

The word "Spring" is written in a large, bold, green, sans-serif font. It is centered on the slide and overlaid on a large, faint circular seal of the ISTAD. The seal features a central atomic symbol, binary code (01010100 01000001 01001001) around the perimeter, and the text "INSTITUTE OF SCIENCE AND TECHNOLOGY ADVANCED DEVELOPMENT" and "ISTAD" at the bottom.

Getting Started | Core Technologies

Contents

1. Spring Framework Overview
2. Spring Boot Overview
3. Spring Boot Application with Gradle
4. Dependency Injection
5. Spring Inversion of Control (IoC) Container
6. Application Properties
7. Spring Profiles



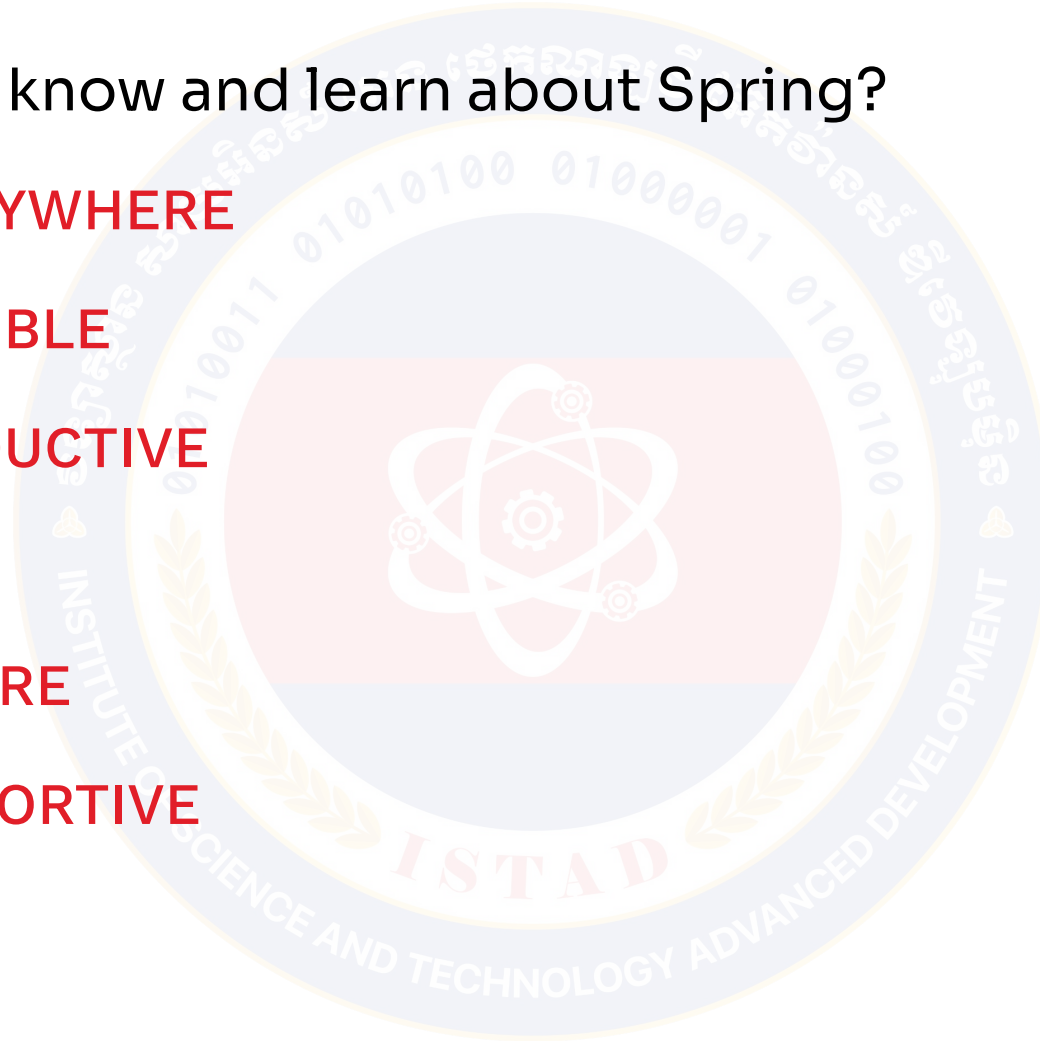
1. Spring Framework Overview

- The Spring Framework is a powerful and flexible framework designed for building Java-based enterprise applications.
- Spring offers a modular and flexible approach to building applications, emphasizing simplicity, code reuse, and testability.
- Spring is known for its vast ecosystem of extensions and integrations, enabling developers to leverage a wide range of additional libraries and tools for various purposes.
- It has a large and active community, providing extensive documentation, tutorials, and support resources for developers.



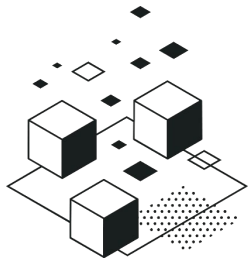
1. Spring Framework Overview (cont.)

- What you need to know and learn about Spring?
 - Spring is **EVERYWHERE**
 - Spring is **FLEXIBLE**
 - Spring is **PRODUCTIVE**
 - Spring is **FAST**
 - Spring is **SECURE**
 - Spring is **SUPPORTIVE**

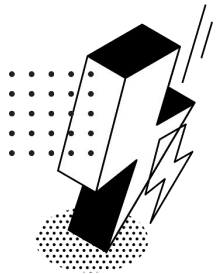


1. Spring Framework Overview (cont.)

▪ What can Spring do?



Microservices



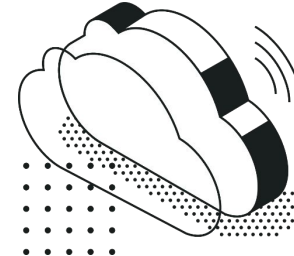
Serverless



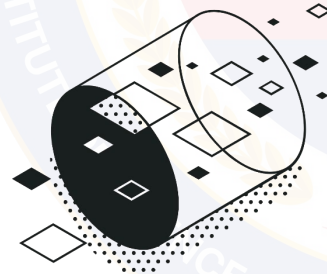
Reactive



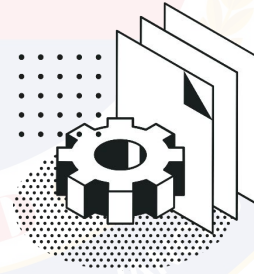
Cloud



Web Apps



Event Driven

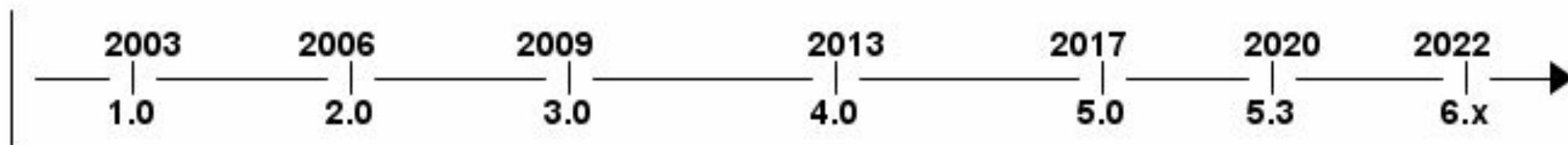


Batch



1. Spring Framework Overview (cont.)

▪ Spring History (Cont.)



1. Spring Framework Overview (cont.)

- Spring History
 - The Spring Framework was created by Rod Johnson, an Australian software developer, in 2002. Johnson published his book titled "Expert One-on-One J2EE Design and Development," which introduced the concepts and ideas that formed the foundation of the Spring Framework.
 - In June 2003, Spring was released under the Apache 2.0 license.
 - The first production release, Spring 1.0, followed in March 2004.
 - Spring 2.0 which came in 2006
 - Spring 2.5, which came in 2007, introduced annotation configurations



1. Spring Framework Overview (cont.)

- Spring History (cont.)
 - Spring 3.0, which came in 2009
 - Spring 3.2, which came in 2012, introduced Java configuration, had support for Java 1.5 to Java 7, Hibernate 4, Servlet 3.0.
 - Spring 4.0, which came in 2014, had support for Java 8.
 - Spring Boot 1.0 arrived in April 2014 as an easy-to-use web application framework based on Spring.
 - Spring 5.0 came out in 2017. Spring Boot 2.0 has support for Spring 5
 - Spring 5.3 (LTS) released in 2020 which the latest version of Spring 5.x
 - Spring Framework 6.0 (November 2022) shifted to a Java 17+ baseline and integrated with Jakarta EE 9+ APIs. Spring boot 3.0 was built on it



1. Spring Framework Overview (cont.)

Spring Projects help developers build an application from simple configuration to security, web apps to big data, whatever the infrastructure needs of your application maybe. It is modular by design.

- Popular Projects: **Spring Boot, Spring Framework, Spring Data, Spring Cloud, Spring Cloud Data Flow, Spring Security, Spring Authorization Server, Spring for GraphQL, Spring Session, Spring Integration, Spring HATEOAS, Spring Modulith, Spring REST Docs, Spring AI, Spring Batch, Spring CLI, Spring AMQP, Spring Flo, Spring for Apache Kafka, Spring LDAP, Spring for Apache Pulsar, Spring Shell, Spring Statemachine, Spring Web Flow, Spring Web Services,...**



2. Spring Boot Overview

- Spring Boot is just an extension of the already existing and expansive Spring framework.
- It has some specific features that make the application easier to create stand-alone, production-grade Spring based Applications that you can “just run”.
- Spring Boot eliminates the need for manual configuration, boilerplate code, and complex setup, allowing developers to focus on writing business logic.
- Spring Boot easily promotes the use of Microservice architecture because of its auto configuration, embedded servers and simplified dependency management.



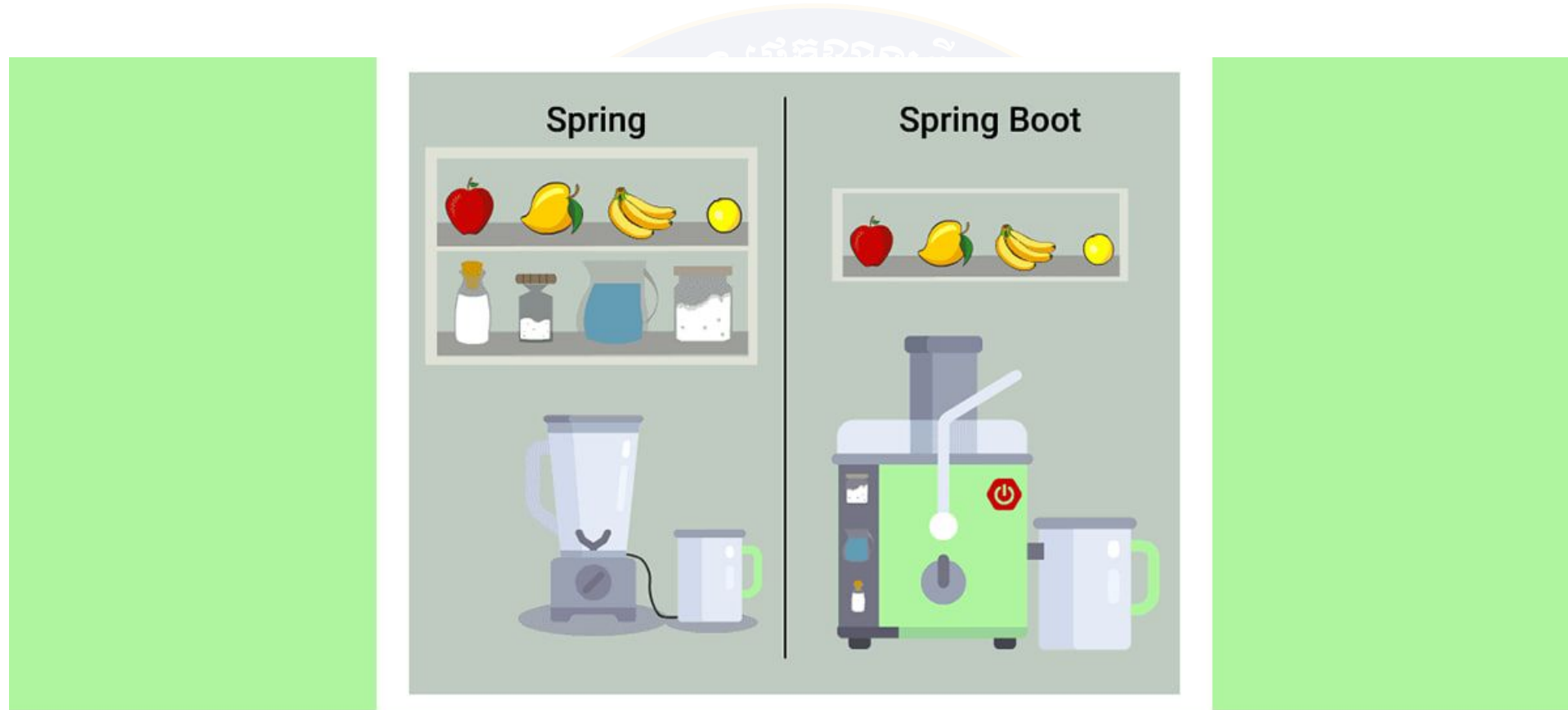
2. Spring Boot Overview (cont.)

- Key features and benefits

Feature	Benefit
Standalone Application	Build the application jar and run without customize deployment
Embedded Servers	Comes with prebuilt application servers such Tomcat, Jetty, ...
Auto-Configurable	Other 3rd party frameworks will be configured automatically
Production-Like Features	Health checks, metrics and externalized configurations
Starter Dependencies	Simplify the build configuration for Maven and Gradle



2. Spring Boot Overview (cont.)



The Difference Between Spring vs. Spring Boot



2. Spring Boot Overview (cont.)

- How does Spring Boot work?
- How does Spring Boot have auto configurations?
- It really comes down to three simple Spring Boot annotations that provide this feature:

@SpringBootApplication

@EnableAutoConfiguration

@ComponentScan



2. Spring Boot Overview (cont.)

- **@SpringBootApplication** is used in the entry point of the application add the class it resides in needs to have to the application main method. The annotation is needed and will provide each of the two annotations to your Spring Boot application since the **@SpringBootApplication** includes both inside.
- **@EnableAutoConfiguration** provides each of the representing class with the Automatic Configuration capability.
- **@ComponentScan** will at initialization scan all the beans and package declarations.



2. Spring Boot Overview (cont.)

- Presently, the latest version of Spring Boot is 3.x.x
- **Highlights of the new release include:**
 - Minimum Java 17 to Java 24
 - Support for generating native images with GraalVM, superseding the experimental Spring Native project
 - Improved observability with Micrometer and Micrometer Tracing
 - Support for Jakarta EE 10 with an EE 9 baseline
 - Support for Virtual Threads
 - RestClient and JdbcClient



3. Spring Boot 3.x.x with Gradle

- Requirements:
 - Favorite text editor or IDE
 - JDK 17 or later
 - Gradle 7.x
- You can create project by using website or IDE
 - Website: <https://start.spring.io/>
 - IDE: IntelliJ IDEA, Spring Tool Suite (STS), VSCode, ...



4. Dependency Injection

What is a dependency?

- Example:
 - When you want to execute the appropriate SQL query, your class would need a database connection.
 - In the Java world, you would get that database connection from another class.
 - We could say that, your class obviously depends on a valid database connection to fire those SQL queries.
 - Totally, your class needs to get the database connection dependency.



4. Dependency Injection (cont.)

How to manipulate dependencies:

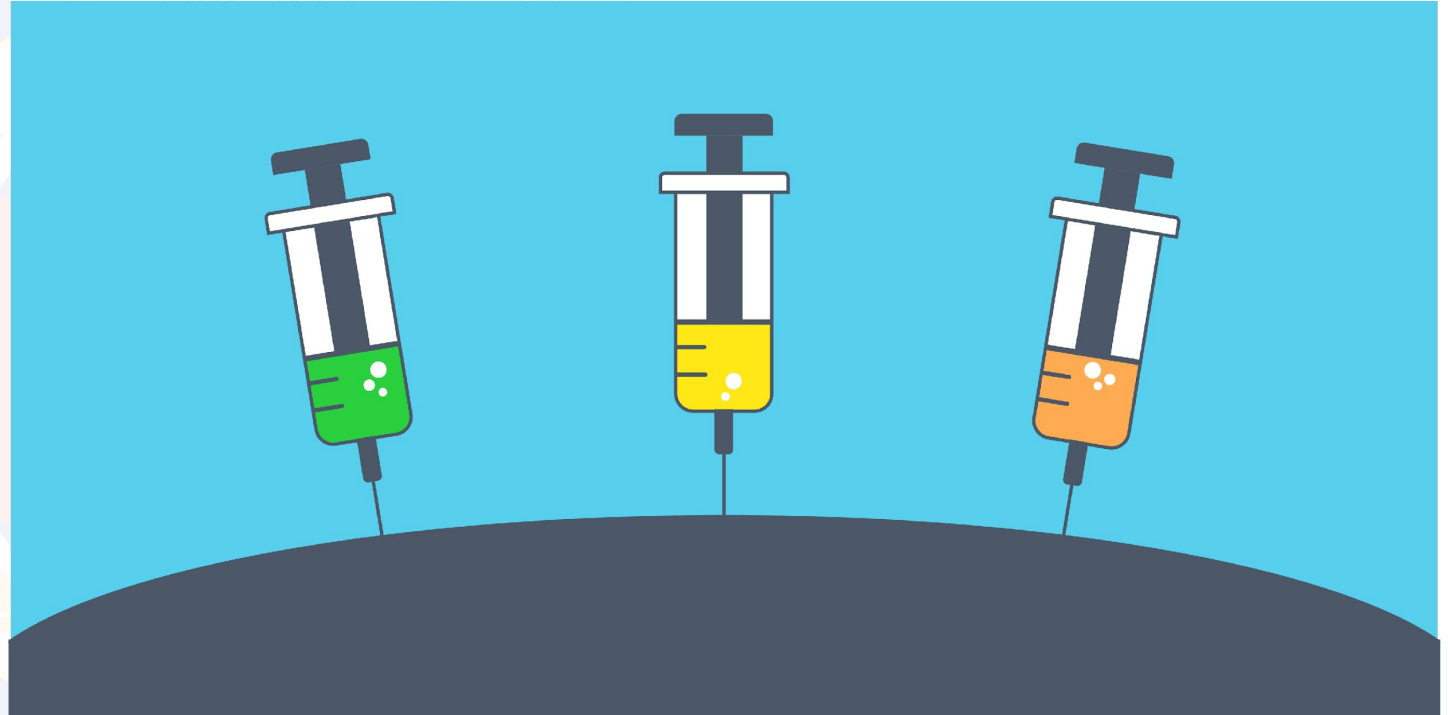
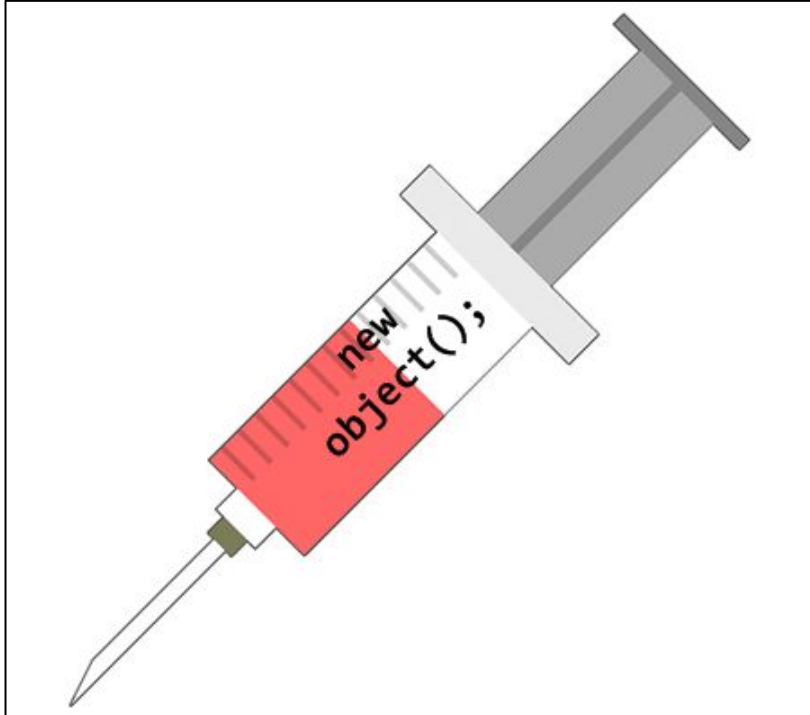
- You can create them in your class with a constructor or method.
- You can manage them in a global application class by using Singleton pattern.

There are however still several drawbacks to these solutions above if your program **gets bigger**, and you **get more and more dependencies**.



4. Dependency Injection (cont.)

Example:



5. Spring IoC Container

What is Inversion of Control (IoC)?

- Inversion of Control is the process of finding the dependencies when needed, but has no control anymore when/how/where it gets from.
- IoC is also known as dependency injection (DI) which is a process whereby objects define their dependencies.
- Spring framework implements the Inversion of Control (IoC) principle as its core by creating a **Spring IoC container** (also called Dependency Injection Container) to solve the complexity of Java development.



5. Spring IoC Container (cont.)

- Spring IoC Container is a core component of the Spring framework which is responsible for managing application beans (object) such as:
 - Instantiation
 - Dependency Injection
 - Lifecycle Management
 - Metadata



5. Spring IoC Container (cont.)

What is a Spring Bean?

- Spring Bean is an object or instance that is instantiated, assembled, and managed by a Spring IoC container.
- Anything can be a Spring Bean if it's configured to be initialized via container by providing configuration metadata information.
- We also could specify how many instances of a bean should Spring creates and lifecycle by using **scopes**.



5. Spring IoC Container (cont.)

What is a Spring Bean?

- Spring Bean is an object or instance that is instantiated, assembled, and managed by a Spring IoC container.
- Anything can be a Spring Bean if it's configured to be initialized via container by providing configuration metadata information.
- We also could specify how many instances of a bean should Spring creates and lifecycle by using **scopes**.



5. Spring IoC Container (cont.)

Spring Bean Scopes

- There are **5 scopes** defined for Spring Beans:
 1. **singleton** – only one instance of the bean will be created for each container (*default*)
 2. **prototype** – a new instance will be created every time the bean is requested
 3. **request** – the same as prototype scope, however it's meant to be used for web application. A new instance of the bean will be created for each HTTP request
 4. **session** – a new bean will be created for each HTTP session by the container
 5. **global-session** – is used to create global session beans for Portlet application



5. Spring IoC Container (cont.)

- There are **3 ways** to configure beans to be used in the application:
 - **XML-based configuration** – by using XML files (*legacy*)
 - **Annotation-based configuration** – by using **@Service**, **@Repository**, **@Component**, **@Controller**, **@RestController** and scope details can be provided with **@Scope**
 - **Java-based configuration** – by creating Java configuration class annotated with **@Configuration**, then define beans using **@Bean** on methods within the configuration class.



5. Spring IoC Container (cont.)

- Example of Annotation Based Spring Bean Configuration

```
@Service
public class MyAnnotatedBean {

    private int number;

    public int getNumber() {
        return number;
    }

    public void setNumber(int number) {
        this.number = number;
    }

}
```

MyAnnotatedBean is configured using **@Service**



5. Spring IoC Container (cont.)

- Example of Java Based Spring Bean Configuration

```
@Configuration
public class AccountBeanConfig {

    @Bean
    Account getAccount() {
        return new Account("ACT-001", "5000");
    }

    @Bean
    Account getAccount() {
        return new Account("ACT-002", "1000");
    }
}
```



5. Spring IoC Container (cont.)

Dependency Injection – DI

- There are **3 ways** to inject dependency:
 1. **constructor-based injection** – Recommend and best practice
 2. **setter-based injection** – Utility or supporting dependencies
 3. **field-based injection** – Test module only



5. Spring IoC Container (cont.)

Dependency Injection – Useful Annotations

@Autowired – mark a dependency which Spring is going to resolve and inject.

We can use it with a setter or field injection.

```
// Example
```

```
private MyAnnotatedBean myAnnotatedBean;
```

```
@Autowired
```

```
public void setMyAnnotatedBean(MyAnnotatedBean myAnnotatedBean) {  
    this.myAnnotatedBean = myAnnotatedBean;  
}
```



5. Spring IoC Container (cont.)

Dependency Injection – Useful Annotations

@Qualifier – along with **@Autowired** to provide the bean id or bean name we want to use in ambiguous situations.

```
// Example
@Autowired
@Qualifier("myBean")
public void setMyAnnotatedBean(MyAnnotatedBean myAnnotatedBean) {
    this.myAnnotatedBean = myAnnotatedBean;
}
```



6. Application Properties

- **Application Properties** is a file which allows to define various configuration properties for Spring application.
- It allows us to:
 - Customize behavior
 - Setup connection (socket, kafka, database, mail, ...)
 - Component or bean configuration
- By default, Spring Boot looks for an **application.properties** file in the classpath root.
- We can also use YAML format by renaming to **application.yaml**
- **.properties** and **.yaml** have different syntax, that's why you need to understand and recognize it carefully.



6. Application Properties

- Key differences

Property	Application Properties	Application YAML
Format	Key-value pairs	Human-readable
Readability	Less readable	More readable
Type Support	Strings only	Multiple data types
Naming Conventions	Dots (.)	Indentation
Spring Support	Supported	More intuitive
File Extension	.properties	.yaml

<https://env.simplestep.ca/> (Testing)



6. Application Properties

Read value with @Value

- These properties can be accessed in your application code using Spring's @Value annotation or through the Environment object.

```
@Value("${paypal.password}")  
private String password;
```

```
private PaymentService(@Value("${paypal.url}") String paypalUrl) {  
    this.paypalUrl = paypalUrl  
};
```



7. Spring Profiles

- Spring Profiles allow you to define different configurations for different environments (e.g., development, staging, production).
- Profiles are typically defined in the **application.properties** or **application.yaml** files.
- Define separate property files for each profile:
 - **application-dev.properties**
 - **application-prod.properties**
- Spring Boot will automatically load the relevant properties based on the active profile (**spring.profiles.active**).

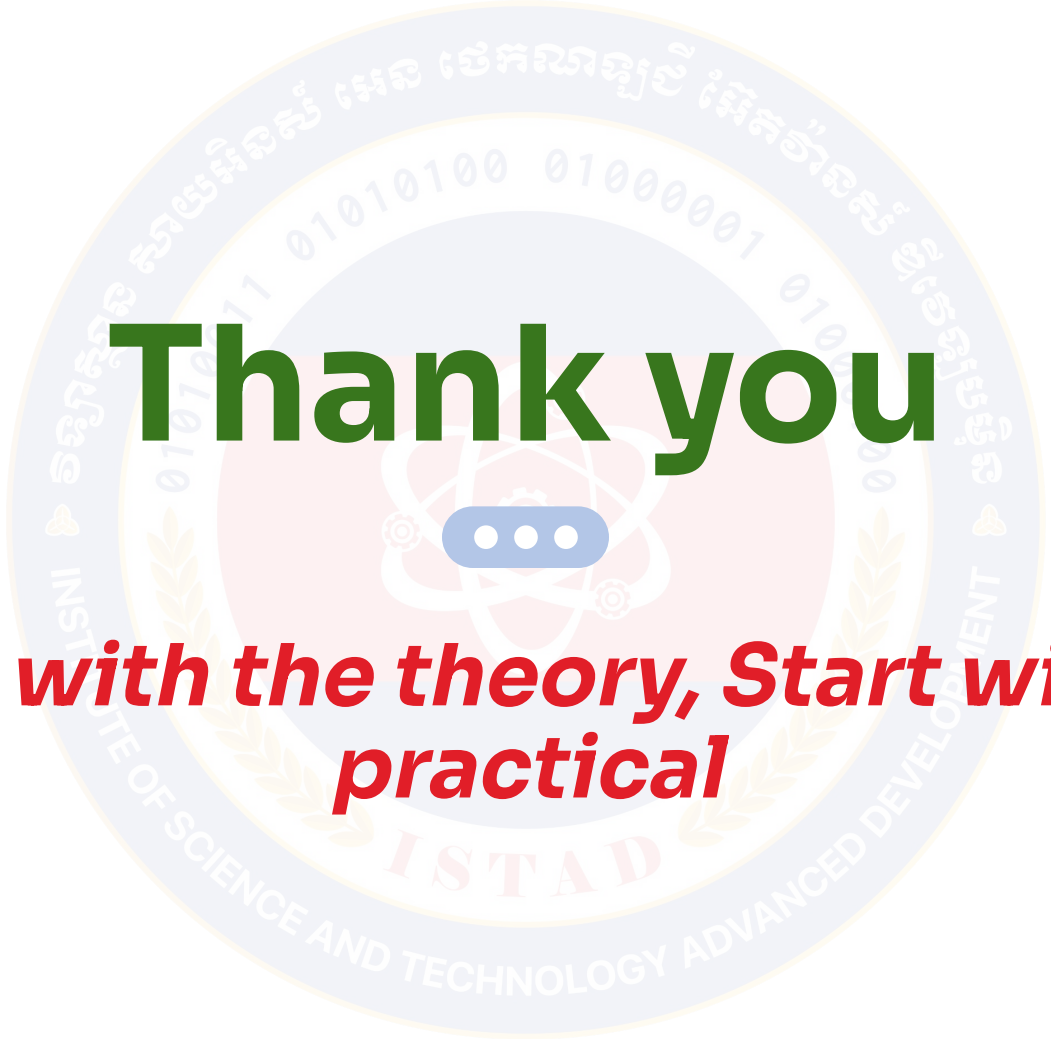


7. Spring Profiles (cont.)

Example

```
spring:
  application:
    name: my-application
  datasource:
    url: jdbc:postgresql://localhost:5432/mydb
    username: dbuser
    password: dbpass
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  custom:
    feature-flags:
      new-feature-enabled: true
    cache:
      ttl: 3600
    api:
      base-url: https://api.example.com/v1
      timeout: 5000
```



A large, faint watermark of the ISTAD seal is centered in the background. The seal is circular with a blue border containing the text 'INSTITUTE OF SCIENCE AND TECHNOLOGY ADVANCED DEVELOPMENT' and 'ISTAD' in the center. Inside the seal, there is a stylized atomic symbol and a blue speech bubble with three white dots.

Thank you

Begin with the theory, Start with the practical